

一条SQL导致的系统异常：死锁问题怎么解决？

一 背景

最近线上消费 RocketMQ 的服务频繁报 SQL 死锁异常，虽然最终可以基于事务自动回滚和逻辑重试保证最终正确性，但若一直放任不管，海量报警日志会掩盖真正需要紧急处理的异常，同时频繁回滚也会降低消费端的吞吐量。个人通过分析线上服务日志、MySQL 死锁日志、梳理 MySQL 在 RR 级别下的锁机制，找到了真正的问题所在，并对业务处理逻辑进行了优化

二 知识储备

正所谓“工欲善其事，必先利其器”，在具体介绍 CASE 背景和解决方案前，先对需要系统了解的知识点进行详细介绍，以便大家能够快速理解解决方案。

死锁通常是因为两个及以上事务发生了死循环锁依赖，此时不得不回滚来释放锁，那么事务是个什么东西？

1 事务

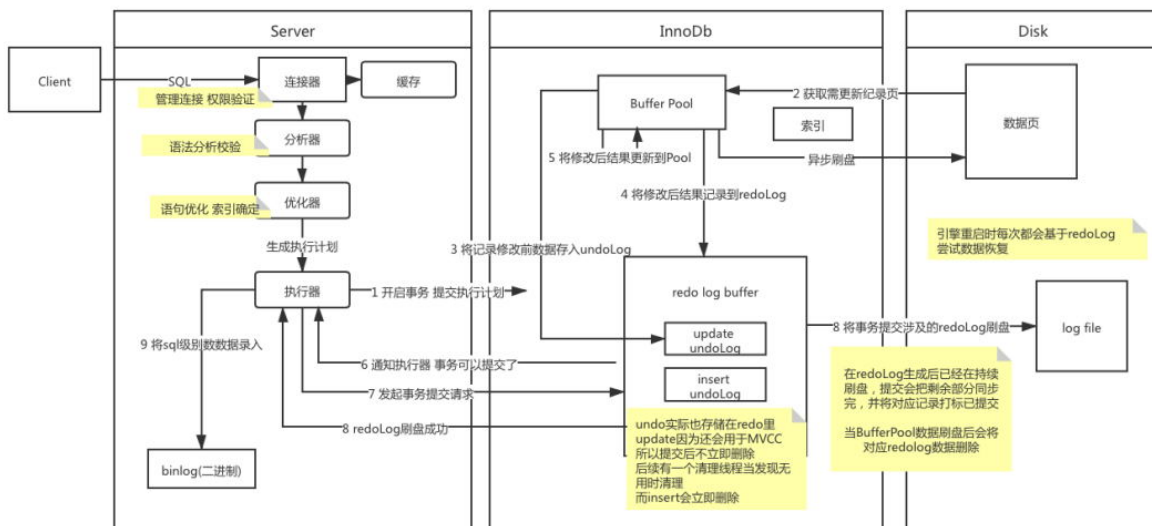
为什么需要事务？

我们在业务实现时，经常需要保证某一批 SQL 能够具备 ACID 特性，如果没有事务，在应用里自己保证将会变得非常复杂，InnoDB 引擎引入事务机制，极大简化了我们在此方面的编程模型。

ACID 的实现机制是什么？

- 原子性 (Atomicity)：事务内 SQL 要么同时成功要么同时失败，基于 UndoLog 实现。
- 一致性 (Consistency)：系统从一个正确态转移到另一个正确态，由应用通过 AID 来保证，并非数据库的责任。
- 隔离性 (Isolation)：控制事务并发执行时数据的可见性，基于锁和 MVCC 实现。
- 持久性 (Durability)：提交后一定存储成功不会丢失，基于 RedoLog 实现。

下面简单说下 RedoLog、UndoLog 在整个执行过程中的流程：



为什么需要 UndoLog？

InnoDB 为支持回滚和 MVCC，需要旧数据存档，UndoLog 就负责存储这些数据，当更新 BufferPool 数据前，先将之前数据存入 UndoLog。

为什么需要 RedoLog?

BufferPool 是随机 IO 以页为单位，性能损耗很大，不可每次提交都同步刷盘，需要后续异步进行。不能同步刷就会有一个问题，如果 MySQL 宕机，而事务已提交在 BufferPool 的数据还没有刷到磁盘，就会导致数据丢失持久性无法保证。为此引入 RedoLog，这个文件 IO 是顺序追加 IO 且以修改为单位，性能很高，每次事务提交持久化 RedoLog 到磁盘也不会对性能造成太大影响，如果宕机可以通过重启从 redoLog 恢复丢失数据。

RedoLog 高性能?

映射一段连续的存储空间，保证顺序 IO，数据先写入 Buffer，后一次性批量将事务数据写入磁盘。

2 锁

下面咱们说说 InnoDB 锁机制（此处重点关注）。

为了控制事务并发时的数据安全，在不同隔离级别下会通过不同的协同机制进行处理。传统隔离机制，完全由锁 (LBCC) 来处理，但是这样只能满足读读并发，会对性能造成很大影响，故而出现了支持读写并发的 MVCC。因为 MVCC 不涉及此次背景，也不想罗列锁各种类型（避免让大家直接晕在这里），就简单直接的列出 update、delete、insert 的加锁情况（RC 和 RR 不一样）。

Update & Delete 语句加锁

1) 聚簇索引（查询命中）

```
UPDATE students SET score = 100 WHERE id = 15;
```

X 锁

Read Committed

15	18	20	30	37	49	50
S0001	S0002	S0003	S0004	S0005	S0006	S0007
Bob	Alice	Jim	Eric	Tom	Tom	Rose

X 锁

Repeatable Read

15	18	20	30	37	49	50
S0001	S0002	S0003	S0004	S0005	S0006	S0007
Bob	Alice	Jim	Eric	Tom	Tom	Rose

RC、RR 都是对聚簇索引加 X 锁。

注：RC 读已提交(READ COMMITTE)、RR 可重复读(REPEATABLE READ)

2) 聚簇索引（查询未命中）

```
UPDATE students SET score = 100 WHERE id = 16;
```

Read Committed

15	18	20	30	37	49	50
S0001	S0002	S0003	S0004	S0005	S0006	S0007
Bob	Alice	Jim	Eric	Tom	Tom	Rose

GAP 锁

Repeatable Read

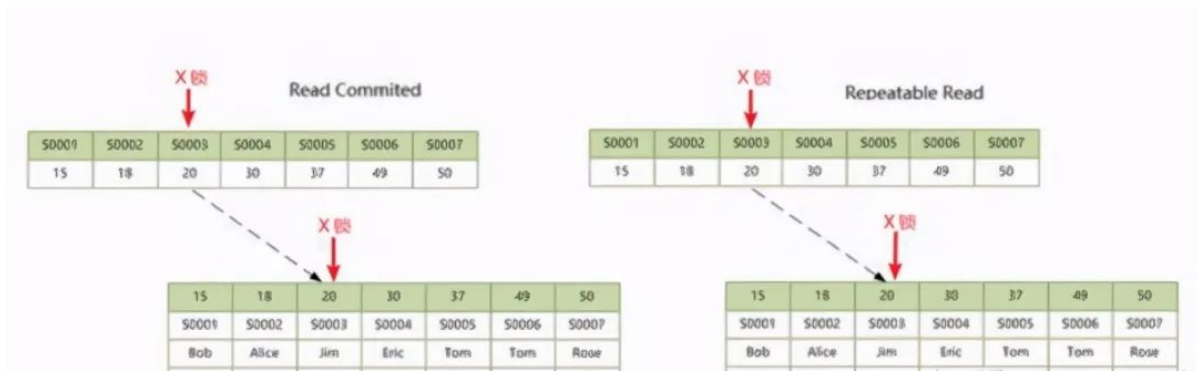
15	18	20	30	37	49	50
S0001	S0002	S0003	S0004	S0005	S0006	S0007
Bob	Alice	Jim	Eric	Tom	Tom	Rose

RC 不加锁，RR 在 16 之前和之后的范围里加 GAP 锁。

注：Gap 是间隙锁，在索引记录之间的间隙中加锁，或者是在某一条索引记录之前或者之后加锁，并不包括该索引记录本身。gap lock 的机制主要是解决可重复读模式下的幻读问题

3) 二级唯一索引（查询命中）

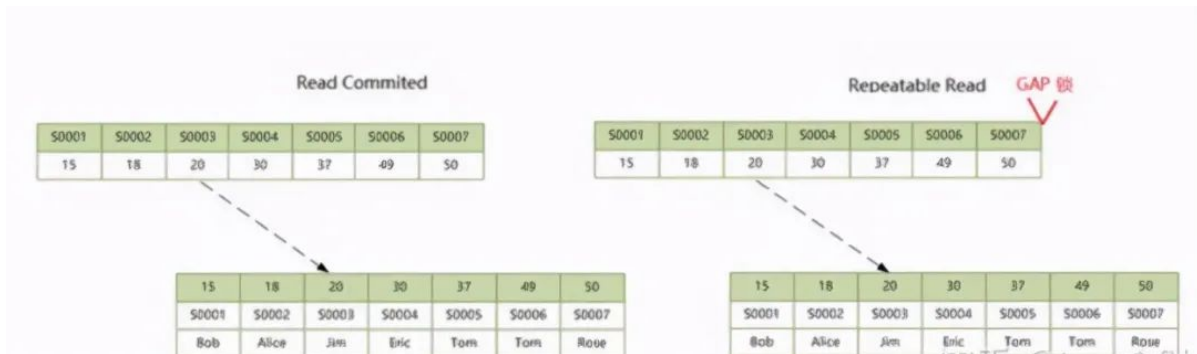
```
UPDATE students SET score = 100 WHERE no = 'S0003';
```



RC、RR 会对二级和聚簇索引都加 X 锁（防止其他事务通过聚簇改数据）。

4) 二级唯一索引（查询未命中）

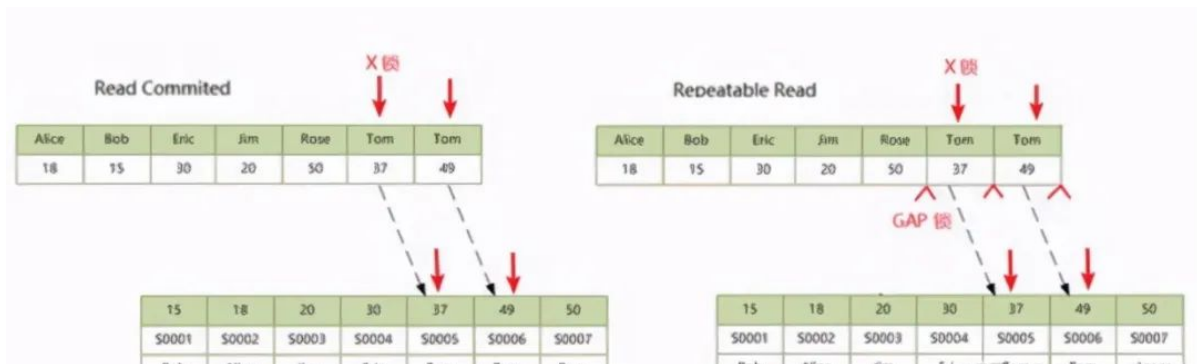
```
UPDATE students SET score = 100 WHERE no = 'S0008';
```



RC 不加锁，RR 只在二级索引加 GAP。

5) 二级非唯一索引（查询命中）

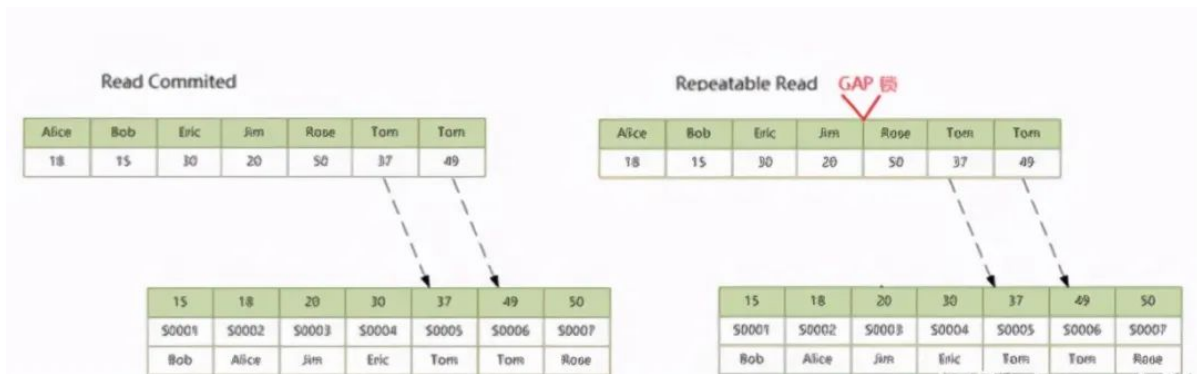
```
UPDATE students SET score = 100 WHERE name = 'Tom';
```



RC 对二级和聚簇加 X 锁，RR 对二级加 X 锁和 Gap 对聚簇加 X 锁。

6) 二级非唯一索引（查询未命中）

```
UPDATE students SET score = 100 WHERE name = 'John';
```



RC 不加锁，RR 只在二级索引加 GAP。

INSERT 语句加锁

- 为了防止幻读，如果记录之间加有 GAP 锁，此时不能 INSERT。
- 如果 INSERT 的记录和已有记录造成唯一键冲突，此时不能 INSERT。

三 线上 CASE

1 分析服务线上日志

发现死锁是两个事务对同一个表先 delete 后 insert 交叉进行引起的：

```
delete from db.table where creativeid=102 (且删除条数为0)
delete from db.table where creativeid=103 (且删除条数为0)
insert into db.table (creativeid) values (102)
insert into db.table (creativeid) values (103)
```

2 分析 MySQL 死锁日志

```
2020-12-14 16:46:55 7fcae52ac700
*** (1) TRANSACTION:
TRANSACTION 9522792935, ACTIVE 0 sec inserting
mysql tables in use 1, locked 1
LOCK WAIT 7 lock struct(s), heap size 1184, 4 row lock(s), undo log entries 2
MySQL thread id 53083906, OS thread handle 0x7fcae44e4700, query id 36945400527 11.1.0.0:3306 huchuan update
insert into db.table (creativeid) values (102)
*** (1) WAITING FOR THIS LOCK TO BE GRANTED:
RECORD LOCKS space id 1001 page no 2323 n bits 560 index 'idx_creativeid' of table 'huchuan.tbl_creative' trx id 9522792935 lock_mode X insert intention waiting
Record lock, heap no 1 PHYSICAL RECORD: n_fields 1; compact format; info bits 0
0: len 8; hex 73757072656d756d; asc supremum;;

*** (2) TRANSACTION:
TRANSACTION 9522792934, ACTIVE 0 sec inserting, thread declared inside InnoDB 5000
mysql tables in use 1, locked 1
7 lock struct(s), heap size 1184, 4 row lock(s), undo log entries 2
MySQL thread id 53083905, OS thread handle 0x7fcae52ac700, query id 36945400528 11.1.0.0:3306 huchuan update
insert into db.table (creativeid) values (103)
*** (2) HOLDS THE LOCK(S):
RECORD LOCKS space id 1001 page no 2323 n bits 560 index 'idx_creativeid' of table 'huchuan.tbl_creative' trx id 9522792934 lock_mode X
Record lock, heap no 1 PHYSICAL RECORD: n_fields 1; compact format; info bits 0
0: len 8; hex 73757072656d756d; asc supremum;;

*** (2) WAITING FOR THIS LOCK TO BE GRANTED:
RECORD LOCKS space id 1001 page no 2323 n bits 560 index 'idx_creativeid' of table 'huchuan.tbl_creative' trx id 9522792934 lock_mode X insert intention waiting
Record lock, heap no 1 PHYSICAL RECORD: n_fields 1; compact format; info bits 0
0: len 8; hex 73757072656d756d; asc supremum;;

*** WE ROLL BACK TRANSACTION (2)
```

可见事务 1 要对一个已被间隙锁控制的记录进行插入意向锁录入，遂进入阻塞等待间隙锁释放，而恰巧另一个事务也同样要对一个被间隙锁控制的记录进行插入意向锁录入，阻塞等待，当两个事务间隙锁碰巧有交集时就进入了死循环最后死锁。

意向锁的存在是为了协调行锁和表锁的关系，用于优化InnoDB加锁的策略。意向锁的主要功能就是：避免为了判断表是否存在行锁而去全表扫描。

意向锁是由InnoDB在操作数据之前自动加的，不需要用户干预；

意向共享锁 (IS锁)：事务在请求S锁前，要先获得IS锁

意向排他锁 (IX锁)：事务在请求X锁前，要先获得IX锁

3 梳理解决方案

- 降低隔离级别为 RC，避免间隙锁（降级后会有不可重复读和幻读问题）。
- 设置 InnoDB 在 RR 级别下不使用间隙锁（关闭后会有幻读问题）。
- 删除前先判断是否存在，存在再删除，可以完全避免死锁（会导致重复数据录入）。

在极端情况下，两个事务同时执行 Select 都不存在然后 Insert，导致重复数据录入。

解决方案：

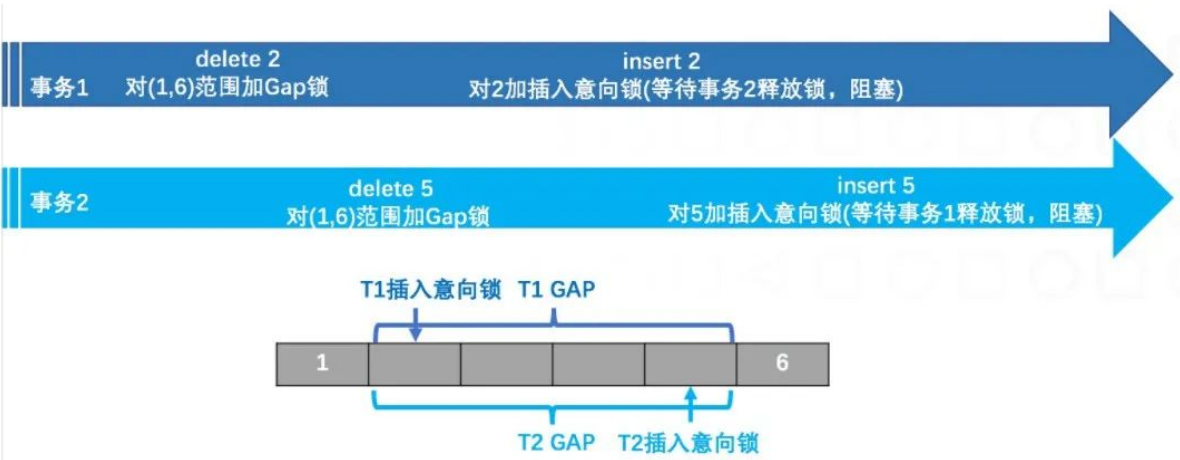
- 方案 1：select for update（会降低并发度）。
- 方案 2：加唯一索引，捕获异常回滚不执行。
- 方案 3：若允许极端少数重复数据（仅文案展示），则无需处理。

另外也要注意尽量避免大事务，它不仅会降低并发还会提高死锁几率。

最终解决方案采用先判断再删除，目前涉及表为文案展示，允许极端情况下少量数据重复，故而暂不做绝对唯一处理。

4 方案 3 原理详解

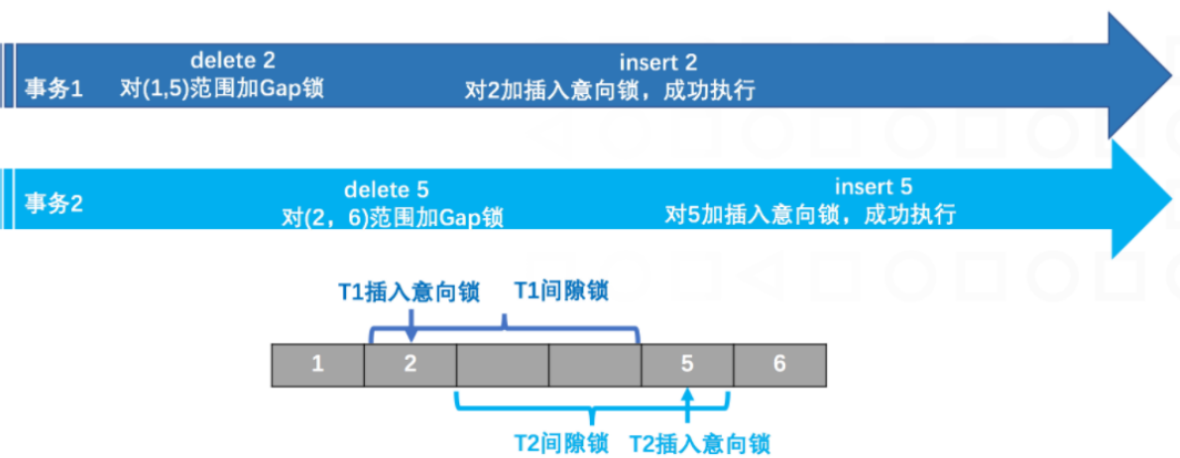
还原线上场景：假设表中有 1，6 两条数据，两个事务分别要对不存在的 2、5 进行先删后插，且交叉执行。



假设表中不存在 2 和 5 对应记录，只有 1 和 6

可见 T1 和 T2 的插入意向锁都要等待对方释放 Gap 锁，死循环。

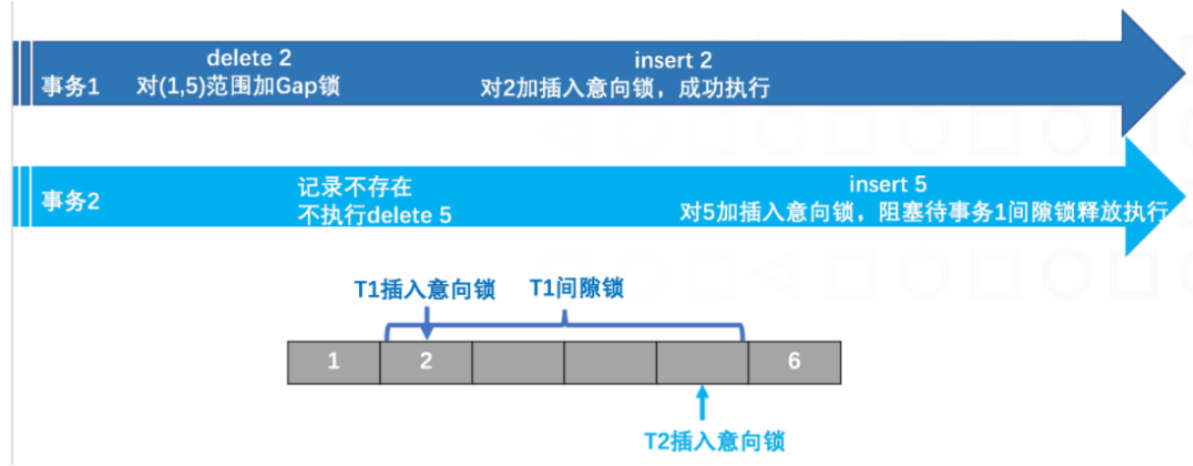
现在我们修改逻辑，在删除前先判断，只有存在记录才进行 delete 操作。



假设表中 2 和 5 都存在

可见事务 1 和事务 2 的间隙锁范围不重叠，都可以成功施加插入意向锁。

我们再罗列另外一种情况，就是 2 或 5 只存在一个，会不会出现死锁呢？



假设表中 2 存在

可见虽然事务 2 可能插入意向锁记录被事务 1 占据，但是不会有死循环发生，等到事务 1 执行完释放锁就可以继续进行了。

综上所述，方案 3 可以完全避免死锁问题。

四 死锁场景分享

死锁案例一

事务 A	事务 B
UPDATE students SET score = 100 WHERE id = 20;	
	UPDATE students SET score = 100 WHERE id = 30;
UPDATE students SET score = 100 WHERE id = 30;	
	UPDATE students SET score = 100 WHERE id = 20;

死锁案例二

事务 A	事务 B
UPDATE students SET score = 100 WHERE id = 25;	
	UPDATE students SET score = 100 WHERE id = 26;
INSERT INTO students (id, no, name, age, score) VALUE(25, 'S0025', 'Sony', 28, 90);	
	INSERT INTO students (id, no, name, age, score) VALUE(26, 'S0026', 'Ace', 25, 88);

25 和 26 记录都不存在，A 和 B 并没有更新任何记录，但是由于数据库隔离级别为 RR，所以会在 (20, 30) 之间加上间隙锁。之后 A 和 B 分别执行 INSERT 要插入 25 和 26，需要在 (20, 30) 之间加插入意向锁，插入意向锁和间隙锁冲突，所以两个事务互相等待，最后形成死锁。

死锁案例三

事务 A	事务 B
UPDATE students SET score = 100 WHERE id < 30;	UPDATE students SET score = 100 WHERE age > 23;

加锁是一条记录一条记录挨个加锁，如果两条 SQL 语句的加锁顺序不一样，也可能会导致死锁。A 的加锁顺序为：id = 20 -> 30，B 的加锁顺序为：id = 30 -> 20，正好相反，所以会导致死锁。

死锁案例四

REPLACE INTO 和 INSERT ON DUPLICATE UPDATE。

这两个语句虽然原子化“存在则更新，不存在则插入”的语义，但在 MySQL 内部还是被拆为多个操作步骤，且在某些版本 (5.7) 会引入 GAP 锁来保证数据完整性，从而导致高并发情况下产生死锁。